

Generating C Code from ForSyDe Models: A Comparison of Traditional Compilation, Template Haskell, and GHC Core

Klara Modin - Michel Delli Abo - Muhammad Afif Ramadhan
Samuel Miksits - Sebastian Thaeron - Zicong Zhang
`{smod,michelda,ramad,smiksits,thaeron,zicong}@kth.se`

November 3, 2025

Abstract

This literature review investigates approaches to compile ForSyDe (Formal System Design) models written in Haskell into C code for embedded software. ForSyDe provides formal system modelling in high-level abstractions that can be translated for hardware, but its application to embedded software is currently limited. Three approaches are evaluated: building a traditional compiler, using Template Haskell for meta-programming, and leveraging the Glasgow Haskell Compiler (GHC) Core intermediate representation (IR). This work explores how these three approaches have been applied by other compilers. The study finds that utilising GHC Core, offers the most suitable strengths, manageable weaknesses, and ease of converting functional models to low-level C code because its IR is already in a procedural form.

1 Introduction

When designing complex systems, it can be helpful to approach the systems from a higher level of abstraction. Models of Computations (MoC) help system designers work at a higher level; they impose restrictions on how a problem or solution can be formulated. While a common requirement in programming languages is to be Turing-complete, this may actually be undesirable in MoCs as it can hurt analysability while not giving any benefit for the intended domain [1]. By having the right restrictions, a MoC can be expressive enough

for it to be useful for a class of problems while making it significantly easier to analyse.

The work presented by Lee et al [2] combines earlier works in MoCs and adds to it what is known as the “Tagged signal model” to formalise and provide comparisons between different types of MoCs. The fundamental idea of a MoC is a system of actors, where it is mathematically described how a computation is performed within an actor. Since a MoC can have multiple actors, it also needs to define how these actors need to run, and exchange information between each other through what is defined as signals. The signals provide mathematically and formally analysable restrictions on how the actors execute with regards to communication mechanism and actors ability to execute concurrently.

Data Flow Models of Computation is described in [3] as a paradigm that is suitable to model systems such as Digital Signal Processing (DSP), where data flows between different steps of computations that can be executed concurrently on parallel hardware. Synchronous Data Flow (SDF) is a model within Data Flow in which the amount of data that is consumed and produced by each actor is fixed and known a priori. An example SDF graph is shown in Figure Introduction, where actors are depicted as blue circles, with directed edges showing the direction of the flow of data and the annotated numbers describing how many tokens are consumed and produced by each actor.

ForSyDe (Formal System Design) is a method-

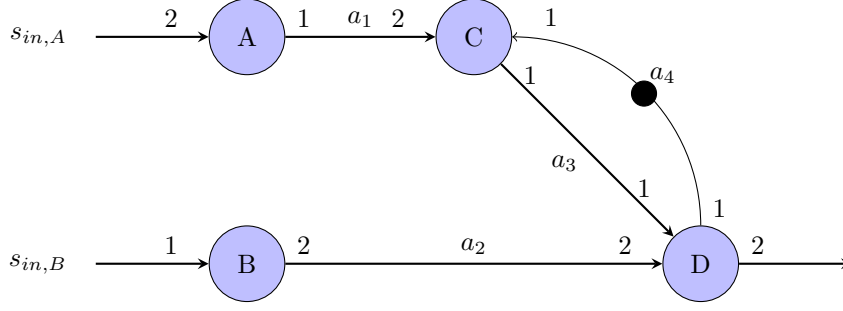


Figure 1: Example SDF Graph

ology for designing a system that is correct-by-construction by emphasizing formal-based high-level modelling and abstraction, and conducting verification early in the design process [4]. Designing with high-level abstractions allows reducing complexity while increasing accessibility when developing for embedded systems. ForSyDe provides the means to implement models at high levels of abstraction through a Haskell program. There have been work to automatically generate a correct implementation for embedded hardware based on models written using ForSyDe, such as ForSyDe-DEEP libraries that generates VHDL.

As mentioned ForSyDe is currently being applied for targeting embedded hardware. However, there is no application of ForSyDe for targeting embedded software. Embedded software is a common use case for systems designers working with microcontrollers and single board computers. Having an application of ForSyDe which generates a correct implementation in a low level programming language would improve the accessibility of developing for embedded systems.

The purpose of this work is to evaluate and compare different approaches for developing a compiler, in order to use in our project that translates ForSyDe models written in Haskell into generated C code. This work will specifically explore three approaches: building a compiler from scratch, utilising Template Haskell for meta-programming, and starting from the Glasgow Haskell Compiler (GHC) Core intermediate representation. The goal is to iden-

tify and compare the strengths and weaknesses of the implementation strategies, in the context of developing tools to apply the ForSyDe framework. The strengths and weaknesses have been identified by conducting a literature review. This review explored works that have applied the three mentioned approaches and developed compilers which start from Haskell.

The outline of the report is as follows: Section Related Work provides a background based on related works. Section Specification describes a set of constraints which minimise the scope of the compiler for the different approaches. Section Approach summarises the three development approaches for implementing a compiler. Section Results describes the different approaches strengths and weaknesses. Section Discussion discusses the results from the previous sections in the context of our intended use case. Section Conclusion summarises our findings and presents the conclusion.

2 Related Work

Lee and Messerschmitt [5] proved that for any valid SDF graph, a static schedule can always be determined. This is accomplished by creating a topology matrix of the data flow graph, where the column represents the node and the rows represent the edges. For each node and edge, a positive number indicates that the node produces that many tokens on the edge, while a negative number indicates that the node consumes that many tokens on the edge. If the rank of this matrix is exactly one less than the

number of nodes, it is schedulable. The null space of the matrix is then the repetition vector, which specifies the number of times each node in the graph has to fire. A schedule can then be derived by simulation, where a node is selected if it has enough input tokens and has not fired as many times as the repetition vector specifies until there are none left.

Courant, Sère, and Shankar [6] developed a code generator for the Prototype Verification System (PVS) specification language into stand-alone C code of similar efficiency as hand written C. They specify a correctness argument for the generator and state that type-safety is preserved in the generated code, and use bi-simulation of source and generated code to verify that the output is the same.

Wehrmeister [7] presented an approach to generate C++ code from UML/MARTE models, targeting the Robotic Operating System framework, with the objective of enabling rapid prototyping and simulation of complex Cyber-Physical Systems.

Silvia, Angelo and Atif [8] proposed a methodology to generate C++ code from Abstract State Machine models supported by Asm2C++, with automatic validation of syntactic and semantic correctness.

Lustre is a data flow language which follows the Synchronous model of computation. Models written using Lustre are transformed into Finite State Machines (FSM) [9]. The Lustre compiler is able to translate these FSMs into generated C code which can be compiled using a standard C compiler and run bare metal.

3 Specification

The compiler should be able to translate SDF models into C code. The SDF models should be defined in Haskell using the ForSyDe Shallow library. The library provides 16 general actors and a delay actor for SDF models. The input and output tokens of the actors are limited to integers. The generated C code should target an embedded board using a Pico 2 mi-

crocontroller. The compiler should at least support translating Haskell functions using addition, subtraction, multiplication, and negation operators. The chosen development approach should ensure the compiler is scalable. It should not be more complicated to add support for additional models using ForSyDe Shallow or Haskell language features.

4 Approach

Each of the explored approaches has a different starting point from which a compiler pipeline would be developed. This is best demonstrated through Figure Compiler Pipeline. A traditional compiler starts from the beginning of a compiler pipeline, requiring custom lexing, parsing, and early intermediate representation transformation passes. Making use of Template Haskell starts the compiler implementation at a well-defined abstract syntax tree (AST). Lastly, reusing the GHC Core IR starts the compiler development from an early intermediate representation. The main difference is in the implementation of the frontend of the compiler.

The later stages of the compiler pipeline, the backend of the compiler, remains the same regardless of the chosen approach. This backend would include an intermediate representation which describes an SDF model written in Haskell as a net list. We call this the ForSyDe IR, which is comprised of a list of process constructors, signals, and functions which the constructors execute. It also includes the global inputs and outputs of the net list. The next step is to transform the ForSyDe IR into another intermediate representation with a procedural form, closely matching the target language of the compiler. In this case, called Procedural IR, which can then be directly transformed into C code.

4.1 Traditional Compiler

The report “Implementing Functional Languages: a tutorial” by Jones and Lester [10] was used to explore the strengths and weaknesses of

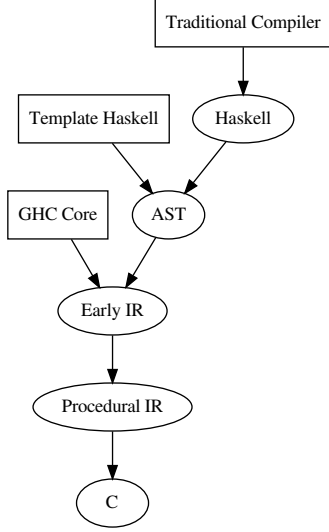


Figure 2: Compiler Pipeline

developing a traditional compiler from scratch. This approach involves implementing a traditional compiler frontend from scratch. This would include developing a lexer and parser for Haskell which results in AST. This frontend can only be developed after identifying and understanding the grammatical rules for at least a subset of Haskell and the ForSyDe framework.

After establishing the compiler frontend, a normalization stage should be performed. The purpose of this stage is to have a simple and refined structure of the original AST in order to have a faster and reliable code execution in the following stages [10].

4.2 Template Haskell

The research report “Hardware Synthesis in ForSyDe” by Woidit [11] was used to explore the strengths and weaknesses of using Template Haskell. Template Haskell is a Haskell extension which allows for type-safe compile-time meta-programming [10]. The extension allows for converting back and forth between Haskell’s concrete syntax and abstract syntax. The features that Template Haskell provides allow for lifting Haskell programs or expressions into an AST [12]. From this point the AST can

be manipulated, changed, or transformed into another intermediate representation. ForSyDe Deep makes use of Template Haskell in its implementation of translating Haskell into a hardware description language [11].

4.3 GHC Core

The research reports “CLasH: From Haskell To Hardware” by Baaij [13] and “Haskell as a higher order structural hardware description language” by Kooijman [14] were used to explore the strengths and weaknesses for developing a compiler starting from GHC Core. Core is a language based on polymorphic lambda calculus used as the intermediate representation of GHC [15]. Translating Haskell to Core is performed using the GHC compiler pipeline, which takes Haskell source code and converts it to Haskell AST, then uses a desugarer. Desugaring is the process of transforming high-level language syntactic sugar to simpler and easier representations, such as Core. GHC performs simplifications and optimizations on the Core representation. Then Core is further converted to other intermediate representations STG and Cmm, and finally compiled into assembly code or other back-ends.

Clash [14] is a tool that utilizes Core, in order to generate hardware description languages. This approach reduces the burden of implementing a compiler frontend from scratch.

5 Results

This section lists the strengths and weaknesses of the Traditional Compiler, Template Haskell, and GHC Core approaches for developing a compiler which starts from models written in Haskell. Each of these approaches and their related works are described in Section Approach.

5.1 Traditional Compiler

Strengths

Full control of how language features specific to Haskell are handled. This includes specifying

restrictions on the program input and ignoring unneeded information at an early stage.

The principles used by a traditional compiler for a functional programming language include graph reduction, lambda lifting, and managing states [10]. These concepts are all relevant when attempting to convert higher-order functional code into first-order C code.

Weaknesses

The usage of intricate algorithms such as lambda lifting [10] increases the complexity of the project. Designing a compiler from scratch is also a lengthy and labour-intensive procedure.

Scaling to support all Haskell language features would be a intensive task. It would involve updating each step of the compiler pipeline and ensuring that each compiler technique being used can handle the features being supported.

5.2 Template Haskell

Strengths

The Template Haskell extension provides direct access to the Haskell AST. It allows a developer to control the structure of a program being translated. Such as parts of the AST being restructured before transformation into an intermediate representation. Template Haskell also allows for the evaluation of a manipulated AST at compile time. [12]

Weaknesses

Due to using the internal structures of the Haskell AST, this approach is highly dependent on the version of GHC used. As for the example with ForSyDe-Deep, it only directly support GHC 7.10.3 and 8.0.1 [11].

Another limitation with Template Haskell is that it is not inherently type-safe [11]. Types can be added, but this needs additional annotation as it does not have access to the Haskell inferred types.

5.3 GHC Core

Strengths

GHC Core reduces the burden of designing a frontend from scratch for Haskell. It is a simple language with significantly fewer language constructs than Haskell, and it is also a type-explicit intermediate representation of Haskell [10].

The pipeline from Haskell to Core is maintained by the GHC team and backed by APIs to perform code parsing, typechecking, normalising, and desugaring [16].

Core transforms the functional form of Haskell into a more procedural form, facilitating a much easier conversion from Core to C code than from Haskell to C code.

The Core specification is also relatively set, and is not intended to change lightly. The stability is desirable as it makes it easier to support multiple versions of GHC.

Weaknesses

Even simple examples of Haskell code result in very verbose Core output. This can cause the initial implementation of the backend to become very complicated even if a lot of programming restrictions are enforced.

Although the Core language does not get changed often and is stable, this does not mean that for every future update to GHC that the same Haskell code will result in the same Core output. This can harm the maintainability of the project.

6 Discussion

According to Results, the AST provided through Template Haskell lacks type information by default. Including types in the AST requires providing explicit type signature annotations. The C language is a typed language, meaning that compiling into C is heavily reliant on information relating to types.

On the other hand, GHC Core is an explicitly typed intermediate representation. Meaning it solves the main weakness of using Template Haskell AST. Creating a compiler based on Core would rely on the GHC API, a library which allows a program written in Haskell to interact with the different components of the GHC compiler. This opens up the ability to reuse GHC's frontend, such as the argument parsing, lexer, and parser. The major benefit of this approach is systematic normalisation of Haskell into Core. Since Core was designed to be an intermediate representation of Haskell it inherently supports synthesising all Haskell language features. This means that a compiler based on Core would already of the foundation necessary to support additional Haskell language and ForSyDe model features. This process eliminates high-level features of Haskell and transforms them into a simpler form using features which are compatible with C. This approach leverages parts of the GHC compiler pipeline, which would have been made manually in a traditional compiler.

7 Conclusion

Developing a ForSyDe compiler with GHC Core as the intended starting point would promote making use of the GHC API, allowing development to focus on the translation from a ForSyDe model to C code, rather than the frontend of a compiler. The weaknesses of using Core can be mitigated by both fixing and limiting the version of GHC and ForSyDe Shallow that can be compiled. This would ensure that the Core intermediate representation stays the same for a provided Haskell program. In conclusion, making use of Core provides a simple yet effective approach to developing a compiler to translate models written using ForSyDe in Haskell to generated C code.

References

- [1] H. Eichelberger, C. Kröher, and K. Schmid, "An Analysis of Variability Modeling Concepts: Expressiveness vs. Analyzability," en, in *Safe and Secure Software Reuse*, J. Favaro and M. Morisio, Eds., Berlin, Heidelberg: Springer, 2013, pp. 32–48, ISBN: 978-3-642-38977-1. DOI: 10.1007/978-3-642-38977-1_3.
- [2] E. Lee and A. Sangiovanni-Vincentelli, "A framework for comparing models of computation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 17, no. 12, pp. 1217–1229, Dec. 1998, ISSN: 1937-4151. DOI: 10.1109/43.736561. Accessed: Sep. 24, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/736561>.
- [3] E. Lee and D. Messerschmitt, "Synchronous data flow," *Proceedings of the IEEE*, vol. 75, no. 9, pp. 1235–1245, Sep. 1987, ISSN: 1558-2256. DOI: 10.1109/PROC.1987.13876. Accessed: Sep. 24, 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/1458143>.
- [4] I. Sander, A. Jantsch, and S.-H. Attarzadeh-Niaki, "ForSyDe: System Design Using a Functional Language and Models of Computation," en, in *Handbook of Hardware/Software Codesign*, S. Ha and J. Teich, Eds., Dordrecht: Springer Netherlands, 2017, pp. 99–140, ISBN: 978-94-017-7267-9. DOI: 10.1007/978-94-017-7267-9_5. Accessed: Sep. 24, 2025. [Online]. Available: https://doi.org/10.1007/978-94-017-7267-9_5.
- [5] E. A. Lee and D. G. Messerschmitt, "Static Scheduling of Synchronous Data Flow Programs for Digital Signal Processing," *IEEE Transactions on Computers*, vol. C-36, no. 1, pp. 24–35, Jan. 1987, ISSN: 1557-9956. DOI: 10.1109/TC.1987.5009446. Accessed: Sep. 24, 2025. [On-

- line]. Available: <https://ieeexplore.ieee.org/document/5009446>.
- [6] N. Courant, A. Séré, and N. Shankar, “The Correctness of a Code Generator for a Functional Language,” en, in *Verification, Model Checking, and Abstract Interpretation*, D. Beyer and D. Zufferey, Eds., Cham: Springer International Publishing, 2020, pp. 68–89, ISBN: 978-3-030-39322-9. DOI: 10.1007/978-3-030-39322-9_4.
- [7] M. A. Wehrmeister, “Generating ROS-based Software for Industrial Cyber-Physical Systems from UML/MARTE,” English, in *IEEE Int. Conf. Emerging Technol. Factory Autom., ETFA*, Journal Abbreviation: IEEE Int. Conf. Emerging Technol. Factory Autom., ETFA, vol. 2020-September, Institute of Electrical and Electronics Engineers Inc., 2020, pp. 313–320, ISBN: 978-1-7281-8956-7. DOI: 10.1109/ETFA46521.2020.9212077.
- [8] S. Bonfanti, A. Gargantini, and A. Mashkoor, “Design and validation of a C++ code generator from Abstract State Machines specifications,” English, *Journal of Software: Evolution and Process*, vol. 32, no. 2, 2020, Publisher: John Wiley and Sons Ltd, ISSN: 2047-7481. DOI: 10.1002/smr.2205.
- [9] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud, “The synchronous data flow programming language LUSTRE,” *Proceedings of the IEEE*, vol. 79, no. 9, pp. 1305–1320, Sep. 1991, ISSN: 1558-2256. DOI: 10.1109/5.97300. Accessed: Sep. 24, 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/97300>.
- [10] S. L. P. Jones and D. R. Lester, “Implementing Functional Languages: A tutorial,” en, Mar. 2000. [Online]. Available: <https://www.microsoft.com/en-us/research/wp-content/uploads/1992/01/student.pdf>.
- [11] H. Woitdt, *Hardware Synthesis in ForSyDe*, eng. 2016. Accessed: Oct. 13, 2025. [Online]. Available: <https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-197307>.
- [12] *Template Haskell - HaskellWiki*. Accessed: Oct. 13, 2025. [Online]. Available: https://wiki.haskell.org/Template_Haskell.
- [13] C. Baaij, “C asH : From Haskell to hardware,” Thesis, University of Twente, Enschede, Dec. 2009. [Online]. Available: <https://purl.utwente.nl/essays/59482>.
- [14] M. Kooijman, “Haskell as a higher order structural hardware description language,” Thesis, University of Twente, Enschede, Dec. 2009. [Online]. Available: <https://purl.utwente.nl/essays/59381>.
- [15] M. Sulzmann, M. M. T. Chakravarty, S. P. Jones, and K. Donnelly, “System F with type equality coercions,” en, in *Proceedings of the 2007 ACM SIGPLAN international workshop on Types in languages design and implementation*, Nice Nice, France: ACM, Jan. 2007, pp. 53–66, ISBN: 978-1-59593-393-5. DOI: 10.1145/1190315.1190324. Accessed: Oct. 20, 2025. [Online]. Available: <https://dl.acm.org/doi/10.1145/1190315.1190324>.
- [16] *Hsc main · Wiki · Glasgow Haskell Compiler / GHC · GitLab*, en, Aug. 2020. Accessed: Oct. 20, 2025. [Online]. Available: <https://gitlab.haskell.org/ghc/ghc/-/wikis/commentary/compiler/hsc-main>.

A Work distribution

- Klara Modin contributed in sections Introduction, Related Work, Approach, and Results. Additionally contributed in terms of the project on example manual implementations of the models, documentation on the target environment, and integrating the testing framework.
- Michel Delli Abo contributed to Related Work, Results, Discussion, and Conclusion. Additionally, acted as a first contact person to the client. Contributed to documentation, translation from Haskell to Core, Core pretty printing, and transformation of Core to another intermediate representation.
- Samuel Miksits - Worked on Introduction, and contributed to Results in the report. Responsible for communicating with external project (Kieler) and contributions to argument parsing and adding example programs to the project.
- Sebastian Thaeon, wrote Section Specification and contributed to Sections Introduction, Results, and Discussion. Contributed to the project through writing documentation, implementing parts of the compiler pipeline, and acting as project manager.
- Muhammad Aff Ramadhan contributed to the AbstractAbstract, and to section Introduction, Results. Additionally contributed to research to project visualisation, documentation.
- Zicong Zhang contributed to section Approach. Additionally contributed in terms of the project on implementation of static scheduling from the ForSyDeIR and corresponding documentations.

All group members participated in project planning, risk analysis, and SEE.

B SEE

This work focuses on exploring approaches to develop a compiler which translates ForSyDe models written using Haskell into generated C code. Such a compiler could potentially cause social and economic turbulence in the industry of systems design. Some positive changes would include reducing development time, improving code quality, and lowering costs associated with software errors. Directly compiling high-level models into C code shortens the time-to-market for embedded solutions, increasing economic sustainability.

On the other hand, the technology poses social and ethical challenges due to potential labour displacement and changes in skill requirements. The efficiency gained from automated code generation could threaten job security for systems engineers, prompting potential industry layoffs, as maintenance and testing requirements diminish. The tool also requires a high level of theoretical knowledge, which may increase inequality between highly-educated and less-educated developers. Despite these challenges, such a compiler could make systems design more accessible field.

C Abbreviations

Abbreviation	Name	Explanation
MoC	Model of Computation	Higher level models on how to describe and implement solutions to problems, typically imposing additional restrictions for greater analysability
SDF	Synchronous Data Flow	A statically schedulable model of computation with bounded buffer sizes
GHC	Glasgow Haskell Compiler	The main compiler for the Haskell language
AST	Abstract Syntax Tree	The output of the lexer and parser
IR	Intermediate Representation	Additional transformations after the AST
ForSyDe	Formal System Design	A framework for designing and implementing embedded systems
API	Application Programming Interface	An explicitly or implicitly defined interface in software, can both be used internally and externally depending on use case
PVS	Prototype Verification System	A specification language
UML/MARTE	UML profile for model-driven development of Real Time and Embedded systems	An extension to UML supporting design and verification/validation of such systems
STG	Spineless, Tagless, G-machine	The next intermediate representation after Core in GHC
Cmm	C--	A version of C-like language which is used as the final IR in GHC